

Towards Verified RCU Reclamation in Asterinas under Relaxed Memory

Haobin Hiroki Chen^{1,3}, Xinyi Wan, Yuwei Liu¹, Sean Anderson², and Lin Huang¹

¹*Ant Group*

²*CertiK Inc.*

³*Indiana University*

Abstract

Conventional formal verification of operating systems often relies on the assumption that the underlying memory model is *sequentially consistent* (SC), under which memory operations appear to execute in a single global order consistent with each core’s program order. While this abstraction can simplify reasoning, it is inadequate for low-level kernel code that makes intensive use of atomic operations and cross-core synchronization primitives, such as barriers and fences. Such mechanisms form the basis of synchronization constructs including read-copy-update (RCU) and atomic exchanges. Verifying this code under SC alone may therefore mask behaviors permitted by weak memory models, leaving subtle concurrency bugs or even system crashes undetected.

There is ongoing verification work on the `ostd` module of the Asterinas kernel, where we use Verus, a deductive verifier for Rust, to establish memory safety for core kernel functionality and expose safe interfaces to external callers. One major missing component, however, is support for weak-memory reasoning. Our current verification largely assumes SC, in part because the necessary weak-memory features have not yet been officially integrated into upstream Verus. This limitation makes it difficult, and in particular prevents us, from verifying the RCU implementation and other synchronization mechanisms whose correctness depends on precise memory-ordering guarantees.

In this report, we present the technical challenges involved in incorporating weak-memory reasoning into the verification of a real-world operating-system kernel, and describe our approaches to verifying RCU and integrating weak-memory semantics into the existing Asterinas verification framework.

1 Introduction

Asterinas [10] is a Rust-based framekernel that aims to provide memory-safety guarantees to external callers. Under the framekernel architecture, the kernel is logically divided into two components: a privileged OS framework, analogous to a microkernel, and a set of de-privileged OS services. The core functionality of the privileged component interacts extensively with hardware and therefore necessarily contains unsafe code. By confining such unsafe code to a small, dedicated *frame*¹, Asterinas substantially reduces the size of its trusted computing base (TCB). Consequently, the overall system can provide stronger security guarantees than conventional kernel designs, provided that the TCB itself is correct.

¹This component is called `ostd`.

The *verified OSTD* project², which we refer to as *vostd* throughout this report, aims to formally verify the *ostd* component using Verus [6], a state-of-the-art deductive verifier for Rust. So far, we have successfully verified the memory-management subsystem and established a foundation for extending verification to additional core kernel components.

RCU (read-copy-update) is an important synchronization mechanism in modern operating-system kernels because it allows readers to access shared data with very low synchronization overhead [7]. Its correctness, however, depends on subtle ordering relationships between memory accesses performed by different cores. In particular, the implementation relies on atomic operations, memory barriers, and carefully chosen memory orderings to ensure that updates become visible to readers at the appropriate points in an execution. These properties cannot be adequately captured by reasoning solely under sequential consistency [12].

This creates a fundamental difficulty for deductive verification. Under a weak memory model, the order in which memory operations become visible to other cores may differ from their program order, and different cores may observe writes in different orders. As a result, a proof must reason not only about the values stored in memory, but also about the synchronization relations that establish when those values may safely be observed. For RCU, this is particularly important when proving that a reader cannot access an object after it has been reclaimed.

Prior work has studied verification under weak memory models [2, 8, 13, 5], largely using (concurrent) separation logic [11, 1] and mechanized in the Iris framework [4, 3], a monoid-based formal system for concurrent program verification written in Rocq and Lean4. In particular, Jung et al. [2] proposed an Iris-based verification framework for RCU-protected concurrent data structures, including stacks and linked lists. Their model represents memory locations using histories indexed by thread-local views, for example as a finite map from thread identifiers to histories, $L : \mathbb{N} \rightarrow \text{Hist}$, while each thread maintains its own logical timestamp or view of memory. These thread-local views can be incorporated into Iris invariants, allowing the proof to track which writes have become observable to each thread. Consequently, concurrent reads and writes can be specified and verified using standard Iris-style Hoare triples, with the invariant capturing the evolution of the underlying weak-memory state.

However, prior work largely stops short of verifying *de facto* systems code. These approaches typically mechanize the underlying program logic, memory model, and safety proofs within a proof assistant such as Rocq, and establish properties such as data-race freedom for programs expressed in the corresponding formal model. As a result, a gap remains between the verified semantics and the actual implementation of a production kernel, including its programming-language semantics, compiler behavior, synchronization primitives, and hardware-facing code. How to bridge these semantic and implementation gaps in the verification of real-world systems software therefore remains an open challenge.

This work. In this work, we take a step toward bringing weak-memory RCU verification into the trusted core of a real Rust operating system. Building on the general-purpose RCU logic of Jung et al. [2], we develop a verification architecture for *vostd* in Verus. Rather than introducing a separate axiomatic model of weak memory, our development connects Verus’s native relaxed-memory semantics to the abstract capabilities used by the RCU logic. Concrete atomic observations, task-local memory views, CPU quiescent states, and delayed callbacks are refined into logical capabilities for reader protection, retirement, grace-period completion, and reclamation.

We use this architecture to verify end-to-end reclamation paths for RCU root pointers and a representative linked-list traversal. The proof connects an acquire load to a valid Rust shared reference, follows an unlinked object through retirement and grace-period detection, and finally recovers its full ownership at a type-erased callback boundary. The development also preserves allocation identities in-

²<https://github.com/asterinas/vostd>

independently of physical addresses, allowing the proof to account for stale weak-memory observations and address reuse.

Our main contribution is therefore not a new RCU algorithm or a replacement for the underlying separation logic. Instead, it is a practical refinement methodology showing how a *general* weak-memory RCU theory can be instantiated using the native memory model, ownership mechanisms, and execution structure of a deductively verified Rust kernel.

2 Background

2.1 Asterinas and vostd

Asterinas [10] is a Rust-based operating system built around the *framekernel* architecture. The kernel is logically divided into a privileged OS framework and a collection of de-privileged OS services. The privileged framework is responsible for low-level functionality such as memory management, interrupt handling, synchronization, and direct interaction with hardware. Because these operations cannot always be expressed using Rust’s safe abstractions, this part of the kernel necessarily contains unsafe code.

OSTD implements this privileged OS framework. Rather than implementing complete OS subsystems, OSTD provides a relatively small collection of abstractions over resources whose misuse could compromise kernel memory safety. These include abstractions for user-mode execution and address spaces, physical memory, tasks and CPU-local state, interrupts and device I/O, as well as synchronization primitives such as `SpinLock`, `Mutex`, and `Rcu`. Asterinas itself is an OSTD client and implements most conventional kernel functionality—including processes, system calls, virtual memory, file systems, networking, scheduling, and device drivers—on top of these abstractions in safe Rust. Thus, the *framekernel* architecture confines the principal runtime trusted computing base for kernel memory safety to OSTD, although the complete system TCB also necessarily includes components such as the Rust toolchain and underlying hardware.

2.2 Rust Atomic Memory Orderings

Concurrent accesses to the shared RCU pointer are performed using atomic operations. Rust atomics follow the C++20 memory model and expose several memory orderings that constrain how atomic operations interact with surrounding memory accesses. Although this model is often referred to as the C/C++11 memory model, it has received subsequent revisions. In particular, C++20 contains changes to the semantics of sequentially consistent operations. These changes are not relevant to the development in this report, which relies on relaxed, acquire, release, and acquire-release operations rather than `SeqCst`.

- **Relaxed.** A relaxed operation is atomic, but imposes no cross-thread ordering on ordinary memory accesses at all. It is appropriate when only the atomic value itself must be synchronized.
- **Acquire.** An acquire operation prevents subsequent memory accesses in the current thread from being reordered before the acquire. More importantly, when an acquire operation reads a value produced by a corresponding release operation, memory writes that happened before the release become visible to the acquiring thread.
- **AcqRel.** `AcqRel` combines acquire and release semantics for read-modify-write operations. Operations such as `swap` or a successful `compare_exchange` may both consume information from an earlier publication and publish a new value.

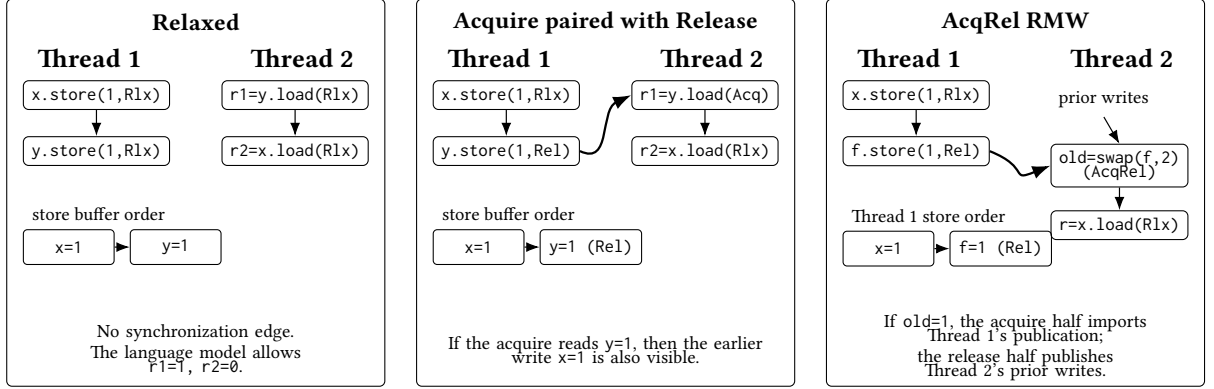


Figure 1: **Effects of the weak memory orderings.** Relaxed accesses do not provide a publication guarantee between data and flag. An acquire load, when reading from a release store, makes the writer's earlier update to data visible. An AcqRel read-modify-write both imports an earlier publication and publishes a new one.

These operations are visualized in Figure 2. These ordering guarantees are important for RCU. Pointer validity alone is insufficient: a reader must also observe the state used to initialize the pointed-to object. Likewise, reasoning about replacement and reclamation must account for the ordering induced by the atomic update.

2.3 Read-Copy-Update

Read-Copy Update (RCU) is a synchronization mechanism designed for workloads in which reads are substantially more frequent than updates. Its main objective is to allow readers to access shared data with very little synchronization overhead. Rather than requiring readers and writers to acquire the same lock, RCU permits readers to continue accessing an existing version of an object while a writer publishes a new version.

An RCU-protected location typically contains a pointer to the current version of some shared object. A reader enters an RCU read-side critical section, obtains the current pointer, and accesses the referenced object. A writer constructs a replacement object and atomically updates the shared pointer so that subsequent readers observe the new version. Readers that obtained the old pointer before the update may continue using the old object. This introduces a central memory-safety requirement: replacing the shared pointer does not imply that the old object can immediately be destroyed. A concurrent reader may still hold a reference to it. RCU therefore separates an update into two logical operations:

- **Publication:** make the new version visible to future readers.
- **Reclamation:** destroy the old version only after all readers that could have observed it have finished.

The interval separating these operations is determined through a grace period.

2.4 Readers, Writers, and Grace Periods

An RCU reader executes inside a read-side critical section. During this interval, the RCU implementation must guarantee that any object obtained by the reader remains valid. Readers usually do not modify the protected object and need not exclude other readers.

A writer replaces the shared pointer but does not immediately reclaim the previous object. Instead, the removed object is placed in a retired state and reclaimed only after a grace period has completed.

A grace period is an interval long enough to guarantee that every reader that might have accessed the old version before its removal has left its read-side critical section. Readers that begin after the update either observe the new version or otherwise satisfy the synchronization conditions imposed by the RCU implementation; they do not prevent reclamation of the retired version indefinitely.

Conceptually, suppose a shared pointer initially refers to object (*A*):

$$P \rightarrow A.$$

A writer creates (*B*) and atomically changes the shared pointer:

$$P \rightarrow B.$$

At this point, (*A*) is no longer reachable through (*P*), but an earlier reader may still hold a reference to (*A*). The writer therefore waits for a grace period:

$$\text{publish } B \longrightarrow \text{grace period} \longrightarrow \text{reclaim } A.$$

The key RCU safety property is therefore:

$$\text{reclaim}(A) \implies \text{no active reader can still access } A.$$

Different RCU implementations detect grace-period completion in different ways. In OSTD, read-side critical sections are tied to preemption control, and the RCU monitor tracks quiescent-state progress across CPUs. A quiescent state indicates that a CPU is no longer executing a read-side critical section associated with the previous grace period. Once every relevant CPU has passed such a state, retired objects may be reclaimed.

This mechanism creates two related proof obligations. First, a reader must access an object that was valid when its pointer was obtained. Second, reclamation must occur only after *every* reader capable of holding the retired pointer has completed. The first obligation concerns pointer publication and memory visibility; the second concerns lifetime and grace-period tracking.

We use the simplified Rust code in Listing 1 to illustrate the design of RCU in `ostd`. An `Rcu<P>` stores an owning pointer of type `P` in raw form inside an atomic pointer. The `NonNullPtr` trait abstracts pointer-owning types, such as `Box<T>` and `Arc<T>`, that can be converted to and recovered from a non-null raw pointer.

Asterinas implements an RCU read-side critical section by disabling task preemption. The read operation first acquires a preemption guard and then loads the protected pointer with acquire ordering. It returns an `RcuReadGuard` containing the observed raw pointer, a reference to the RCU object, and the preemption guard. Retaining the preemption guard keeps the current task inside the read-side critical section for the lifetime of the RCU guard.

Callback-based reclamation. Asterinas does not provide a blocking interface analogous to Linux-like `synchronize_rcu`. Instead, reclamation is performed asynchronously through *callbacks*. After an update removes a pointer from an RCU object, the writer transfers the retired pointer to the global RCU monitor together with a callback that will eventually destroy it. Registering the callback either starts a new grace period, if the monitor is idle, or appends the callback to the batch for the next grace period.

A grace period is advanced at scheduler quiescent points. Since an RCU read-side critical section disables task preemption, the scheduler cannot switch away from a task while an `RcuReadGuard` remains active. Therefore, reaching the task-switching path with preemption enabled constitutes a quiescent

```

1  struct Rcu<P: NonNullPtr> {
2      ptr: AtomicPtr<<P as NonNullPtr>::Target>,
3  }
4
5  struct RcuReadGuard<'a, P: NonNullPtr> {
6      raw_ptr: *mut <P as NonNullPtr>::Target,
7      rcu: &'a Rcu<P>,
8      preemption_guard: PreemptionGuard,
9  }
10
11 impl<P: NonNullPtr> Rcu<P> {
12     pub fn read(&self) -> RcuReadGuard<'_, P> {
13         let preemption_guard = disable_preemption();
14         let raw_ptr = self.ptr.load(Ordering::Acquire);
15
16         RcuReadGuard {
17             raw_ptr,
18             rcu: self,
19             preemption_guard,
20         }
21     }
22 }
23
24 impl<'a, P: NonNullPtr> RcuReadGuard<'a, P> {
25     pub fn get<'b>(&'b self) -> P::Ref<'b> {
26         /* Construct a shared reference from raw_ptr. */
27     }
28
29     pub fn compare_exchange(
30         self,
31         new_ptr: P,
32     ) -> Result<(), P> {
33         /* Replace the pointer if it has not changed. */
34
35         retire(old);
36     }
37 }
38
39 impl Drop for PreemptionGuard {
40     fn drop(&mut self) {
41         decrement_preemption_count();
42     }
43 }

```

Listing 1: Simplified design of RCU in Asterinas. Auxiliary fields and implementation details are omitted.

```
1
2 fn retire<P: NonNullPtr + Send>(old: NonNull<P::Target> {
3     RCU_MONITOR.after_grace_period(move || {
4         let owner = unsafe { P::from_raw(old) };
5         drop(owner);
6     });
7 }
8
9 impl RcuMonitor {
10     fn after_grace_period(&self, cb: Callback) {
11         self.enqueue(cb);
12         self.start_grace_period_if_idle();
13     }
14
15     fn report_quiescent_state(&self, cpu: CpuId) {
16         self.mark_quiescent(cpu);
17
18         if self.all_cpus_are_quiescent() {
19             for cb in self.take_completed_callbacks() {
20                 cb();
21             }
22
23             self.start_next_grace_period_if_needed();
24         }
25     }
26 }
27
28 fn switch(next: Task) {
29     RCU_MONITOR.report_quiescent_state(current_cpu());
30     context_switch(next);
31 }
32
```

state for the current CPU. The scheduler reports this state to the RCU monitor before performing the context switch.

The read guard provides two principal operations. The `get` method constructs a shared reference to the protected object. The lifetime of the returned reference is tied to the borrow of the read guard, so the reference cannot outlive the corresponding read-side critical section. The `compare_exchange` method consumes the guard and uses the pointer observed by `read` as the expected value of the atomic compare-and-exchange operation. If the operation succeeds, the new pointer is installed and reclamation of the old object is deferred until after an RCU grace period. If it fails, ownership of the unused new pointer is returned to the caller.

The `RcuReadGuard` type does not itself implement the Rust Drop trait. Instead, it relies on Rust's structural destruction: when the read guard is consumed or goes out of scope, its fields are dropped, including the embedded `PreemptionGuard`. The destructor of the preemption guard decrements the preemption-disable nesting count. Preemption is enabled again when the outermost preemption guard is released.

2.5 Example: Why RCU Protection Does Not Automatically Extend Through a Linked List

Consider an RCU-protected linked list whose head and internal successor links may be concurrently updated. A reader first enters an RCU read-side critical section and obtains a protected head node. It then repeatedly loads a successor pointer and dereferences the resulting node. A natural but incorrect argument is that, because the reader remains inside the same RCU critical section, every node reached during the traversal is also protected from reclamation. This argument is particularly easy to accept under sequential consistency. Suppose that the list initially contains

$$A \rightarrow B \rightarrow C$$

and that a writer removes *B* by atomically changing *A.next* from *B* to *C*. Under SC, all memory operations belong to a single global order. A load of *A.next* ordered after this update observes *C*, while a load that observes *B* can be ordered before the update. An SC proof may therefore classify every reader that obtains *B* as preceding its removal and conclude that the grace period following the removal is sufficient to protect it.

This reasoning is not valid under relaxed memory. An atomic load need not observe the most recent write in real time. In particular, an acquire load of *A.next* may obtain a value from an older entry in the link's atomic history unless the reader's current view and the synchronization relations rule that observation out.

Consequently, protection of *A* alone does not justify dereferencing the pointer returned from *A.next*. This creates a failure mode that an SC-only verification can miss. An implementation may successfully prove that the source node *A* remains alive while the reader executes, yet fail to establish any protection for the successor *B*. If the proof treats the loaded pointer as immediately dereferenceable, reclamation of *B* may be justified independently even though the reader has obtained a historical pointer to it.

Therefore each traversal step introduces a new proof obligation. After loading an internal link, the proof must (1) relate the selected atomic-history entry to a stable allocation identity, (2) establish that this allocation had not already become unavailable to the current reader, and (3) obtain a physical read permission for that allocation before constructing a Rust reference. RCU protection is therefore extended explicitly from node to node rather than inherited transitively from the root.

This example motivates both parts of our verification problem. Weak-memory reasoning is needed to determine which historical pointer observations are admissible, while the RCU logic is needed to convert an admissible observation into protection against reclamation.

3 Verification Approach

Our goal is to verify the memory safety properties of the RCU-based reclamation in Asterinas under the weak memory model. In particular, we aim to establish that an object removed from an RCU - protected data structure is not reconstructed as an owning Rust ppinter and destroyed while it may still be accessed by an outstanding RCU reader.

We build on the general-purpose RCU logic of Jung et al. [2] since our work does not introduce new theoretic foundation or new Iris semantics for RCU. Instead, we bridge the theoretic and implementational gaps by instantiating the abstract concepts and pure functional logic of the RCU specifications using the concrete execution mechanisms of Asterinas and the native relaxed-memory support available in Verus³.

3.1 Verification Model

We do not introduce a separate operational or axiomatic semantics for relaxed memory. Instead, we take Verus's native IRC11 model as the underlying memory semantics and present an Iris-style logical interface exposing only the observations and resources required by the RCU proof. The notation in this section is therefore an abstraction of the proof state mechanized in Verus, rather than a second memory model.

A *view* is a map

$$V : \text{Loc} \rightarrow \text{Time},$$

where $V(\ell)$ records the latest logical timestamp at location ℓ known to the corresponding execution participant. Views are ordered pointwise:

$$V_1 \sqsubseteq V_2 \triangleq \forall \ell. V_1(\ell) \leq V_2(\ell).$$

Thus, a larger view represents an execution participant that has observed at least as much of the memory history.

Each atomic-history message additionally carries a *message view*. For a history entry

$$H(t) = \langle v, V_m \rangle,$$

the timestamp t orders this write with respect to other writes to the same location, whereas V_m records the observations published with the write. In particular, a release write attaches the writer's current view to the message. If an acquire load later reads that message, its message view is joined into the reader's current view,

$$V'_r = V_r \sqcup V_m, \quad (V_r \sqcup V_m)(\ell) = \max(V_r(\ell), V_m(\ell)).$$

This is how release-acquire synchronization propagates observations across different atomic locations.

For an atomic location ℓ , let its logical history be a finite map

$$H_\ell : \text{Time} \rightarrow_{\text{fin}} \text{Msg},$$

where

$$H_\ell(t) = \langle v, V \rangle, \text{Time} \subseteq \mathbb{N}.$$

means that the per-location logical timestamp t records a write of value v together with the message view V published by that write. For an atomic location ℓ we write

$$\ell \mapsto_{\text{at}} H$$

³although the support is preliminary.

for logical ownership of the atomic location ℓ and its history H .

Each execution participant additionally carries a subjective view describing the latest writes it has observed. We define the ordering between views by

$$V_1 \preceq V_2 \triangleq \forall \ell. V_1(\ell) \leq V_2(\ell).$$

Thus, V_2 covers V_1 when, at every location, V_2 contains an observation at least as recent as that of V_1 . We write $\text{Seen}(V)$ for the persistent assertion that the current execution participant's view covers V .

In particular, a release operation publishes the writer's current view in the message associated with the write, while an acquire operation that observes such a message imports its message view. Schematically, an acquire load satisfies the following Hoare triple

$$\left\{ \text{Seen}(V) \star \ell \mapsto_{\text{at}} H \right\} \quad \text{load}_{\text{acq}}(\ell) \quad \left\{ v. \exists t, V_m. H(t) = \langle v, V_m \rangle \star V(\ell) \leq t \star \text{Seen}(V_m) \star \ell \mapsto_{\text{at}} H \right\}.$$

The condition $V(\ell) \leq t$ expresses the per-location coherence property that we use throughout the RCU proof. Writes to a single atomic location are ordered by their timestamps in that location's history. Once an execution participant has observed the history of ℓ up to timestamp t_0 , a subsequent load from ℓ cannot select an entry at an earlier timestamp: it must select some $t \geq t_0$. This constraint is local to ℓ ; it does not impose a single global order on accesses to different atomic locations. We illustrate this constraint in Figure 2.

Because $\text{Seen}(V)$ is subjective to an execution participant, such state cannot in general be stored directly in a shared invariant. We therefore use the notation $@_V P$ to denote that assertion P holds at a fixed view V . This separates the objective shared facts, such as atomic histories and object metadata, from subjective observations that follow an execution participant. That is, a thread's "local" view is subjective w.r.t. global atomic location's "authoritative", objective facts. Section 4 explains how this distinction is realized by `AtomicPointsTo`, `ViewSeen`, and `RunningTaskContext` in Verus.

Finally, raw addresses are not used as stable object identities in the RCU proof. Each allocation is instead associated with a proof-level allocation identity

$$a \in \text{Ald},$$

and we write $\text{addr}(a)$ for its runtime address. Allocation identity and address are deliberately distinguished because the allocator may reuse the address of a reclaimed object. Hence, for two distinct allocations,

$$a_1 \neq a_2 \quad \text{while possibly} \quad \text{addr}(a_1) = \text{addr}(a_2).$$

This distinction is particularly important under relaxed memory, where an atomic history may retain an observation of an address associated with an earlier allocation. We therefore index ownership, reader protection, retirement, and reclamation resources by allocation identity rather than by raw address.

On top of these weak-memory observations and stable allocation identities, we specify the RCU protocol using logical resources for reader protection, retirement, and reclamation authority, which we introduce next.

3.2 Abstract RCU specification

We now abstract the RCU protocol independently of its concrete realization in Asterinas. The purpose of this abstraction is to characterize the logical resources manipulated by readers, writers, and the grace-period mechanism. Section 4 subsequently shows how these resources are refined into Verus proof state and executable kernel mechanisms.

The notation in this section does not define a separate assertion language. Rather, symbols such as `Guard`, `Retired`, and `ReclaimPermit` name abstract predicates over logical resources in the underlying

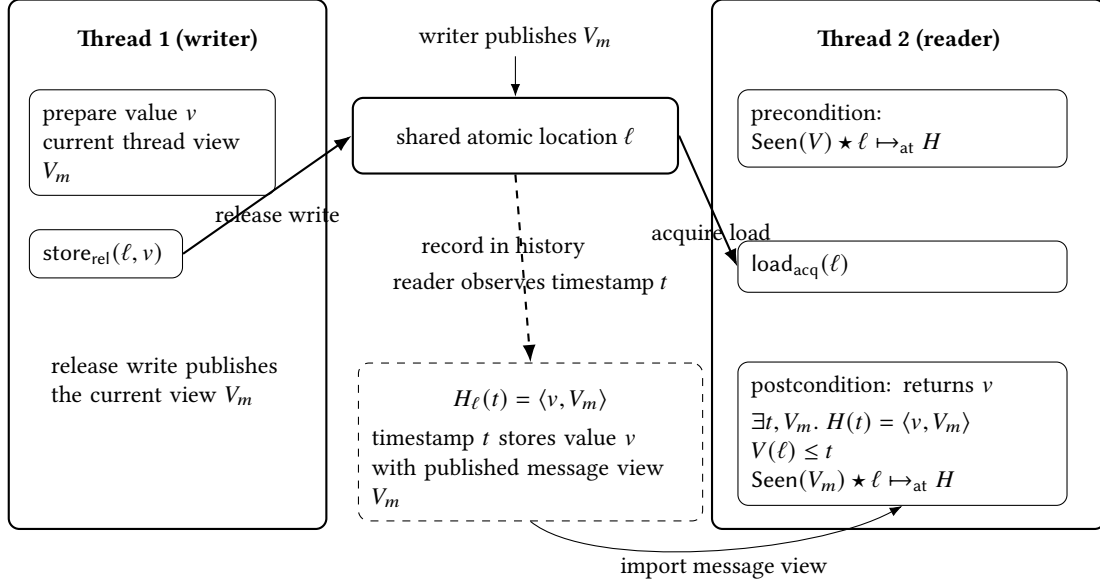


Figure 2: **Informal illustration of the acquire-load rule under the IRC11-style logical interface.** Thread 1 performs a release write that publishes the message view V_m . Thread 2, starting from $\text{Seen}(V)$, performs an acquire load, reads the value v from history entry $H_\ell(t)$, satisfies the coherence condition $V(\ell) \leq t$, and imports the published view $\text{Seen}(V_m)$.

separation-logic presentation. The rules below are derived logical judgments over these predicates. In the mechanization, the corresponding resources are represented by Verus ghost and tracked state, invariants, and native weak-memory proof objects.

Table 1 summarizes the principal assertions used in the abstract specification. We use $\star$ for separating conjunction, $P \vdash Q$ for logical entailment, and $P \mid \Rightarrow Q$ for a frame-preserving logical update.

We do not define a separate inference system for RCU. Instead, the specification is stated using derived rules in an underlying Iris-style separation logic. We assume the ordinary logical and structural rules of that logic, including the rules for separating conjunction, persistence, and frame-preserving updates, and state only the RCU-specific derived rules needed in this development.

We write

$$P \vdash Q$$

for logical entailment. Thus, any proof state satisfying P is sufficient to establish Q . We write

$$P \mid \Rightarrow Q$$

as shorthand for

$$P \vdash \mid \Rightarrow Q,$$

where $\mid \Rightarrow$ is the Iris update modality. Intuitively, this means that the logical state represented by P may be updated in a frame-preserving manner to establish Q . The rules shown below are not intended to be exhaustive; they summarize the RCU-specific transitions used by the subsequent verification.

An RCU reader is represented by the assertion

$$\text{Guard}(t, X, G),$$

Table 1: Principal assertions in the abstract RCU specification.

Assertion	Informal meaning
Seen(V)	The current execution participant has observed at least the memory events described by view V .
BlockInfo(ℓ, a, P)	Runtime address ℓ is associated with allocation identity a and physical resource P .
Guard(t, X, G)	Execution participant t is in an RCU read-side critical section. X records allocations that had already expired when the reader started, and G records allocations currently protected by the reader.
ReadPerm(a, P)	Reader-side permission derived from physical resource P for allocation a .
Retired(a, Q)	Allocation a has been retired together with the observation Q established at retirement.
RemovedObs(ℓ, t_r)	The current view has advanced to at least the removal timestamp t_r at location ℓ .
GPWitness(a, Q)	Abstract grace-period evidence sufficient, together with the retirement fact, to authorize reclamation of a .
ReclaimPermit(a)	Authority to recover the exclusive ownership of retired allocation a for reclamation.

where t identifies the current execution participant, $X \subseteq \text{Ald}$ is the set of allocations that had already expired when the read-side critical section started, and

$$G : \text{Loc} \rightarrow \text{Ald}$$

records the allocations currently protected by the reader. Intuitively, if $G(\ell) = a$, then the reader has established that the value observed at address ℓ refers to allocation a and may continue to access that allocation while the guard remains active.

For each allocated object, we use $\text{BlockInfo}(\ell, a, P)$ to associate a runtime address ℓ with its stable allocation identity a and a physical resource P describing the object. Reader protection may be established only for an allocation that was not already expired when the reader started:

GUARD-PROJECT

$$\frac{a \notin X}{\text{Guard}(t, x, G) \star \text{BlockInfo}(\ell, a, P) \Longrightarrow \text{Guard}(t, X, G[\ell \mapsto a]) \star \text{ReadPerm}(a, P)}$$

Here, $\text{ReadPerm}(a, P)$ denotes the reader-side permission derived from the physical resource P . The concrete mechanization realizes this permission using a linear read lease whose lifetime is tied to the executable RCU guard.

When an allocation is removed from an RCU-protected root, it does not immediately become reclaimable. Instead, we record its retirement using $\text{Retired}(a, Q)$ where Q is an observation established when the object is removed. Under relaxed memory, retaining this observation is essential: a later reader may still observe an older value of the atomic root, and the proof must determine whether such an observation can legitimately refer to the retired allocation.

The expired set X connects the reader's starting view to retirement observations. In particular, if allocation a had already expired when the reader started, then the guard can recover the observation recorded at retirement:

$$\frac{\text{GUARD-SEEN-RETIRED} \quad a \in X}{\text{Guard}(t, X, G) \star \text{Retired}(a, Q) \vdash Q}$$

For an RCU root ℓ , we instantiate Q with a removal observation. Let t_r denote the first timestamp in the root history after allocation a was removed. We write $\text{RemovedObs}(\ell, t_r)$ for the fact that the current view has advanced to at least t_r at location ℓ . Thus, if $a \in X$, GUARD-SEEN-RETIRED implies that the reader's starting view already covers the removal of a . This observation interacts directly with the per-location coherence property. Suppose an acquire load of the RCU root ℓ returns the address of allocation a from a history entry at timestamp t . If the reader's starting view already covers the removal timestamp t_r , then coherence requires

$$t_r \leq t.$$

On the other hand, the root invariant records that every history entry publishing a precedes its removal:

$$H_\ell(t) = \langle \text{addr}(a), V_m \rangle \implies t < t_r.$$

These two inequalities are inconsistent. Hence a valid root load returning a cannot occur when a is already in the guard's expired set.

Retirement alone does not authorize reclamation. We represent the authority to recover an object's exclusive ownership by $\text{ReclaimPermit}(a)$. Such a permit is produced only after the grace period has established that all readers capable of protecting the retired allocation have completed:

$$\overline{\text{Retired}(a, Q) \star \text{GPWitness}(a, Q) \implies \text{ReclaimPermit}(a)}.$$

Note that at this level, GPWitness is abstract and can be treated as opaque in the specification which will be instantiated later.

The central safety property of the protocol is that reclamation authority is incompatible with active reader protection:

$$G(\ell) = a \implies \text{Guard}(t, X, G) \star \text{ReclaimPermit}(a) \vdash \perp.$$

Therefore, once a reclaim permit for a exists, no active reader can retain a valid permission to access the same allocation. The permit may then be consumed by the delayed reclamation callback to recover the object's exclusive ownership.

Putting the rules together. As a small example, consider a reader whose acquire load of an RCU root ℓ returns the address of allocation a . To establish protection for a , we must first show that $a \notin X$, where X is the set of allocations already expired when the reader started. Suppose instead that $a \in X$. By GUARD-SEEN-RETIRED, the reader then observes the removal of a , so its starting view covers the corresponding removal timestamp t_r . If the root load selects the history entry at timestamp t , per-location coherence gives $t_r \leq t$. But the root invariant states that any entry publishing a must satisfy $t < t_r$, a contradiction. Therefore $a \notin X$, and GUARD-PROJECT may extend the guard's protection map with a and derive $\text{ReadPerm}(a, P)$.

Conversely, after retirement and grace-period completion produce $\text{ReclaimPermit}(a)$, the incompatibility between reclamation authority and an active guard prevents any reader from retaining protection for a . Thus the same abstract rules connect a weak-memory observation first to safe reader access and later to safe reclamation.

3.3 Extending RCU Protection Across Atomic Links

Verifying reclamation of an RCU root is not sufficient for verifying RCU-protected data structures. A reader typically obtains an initial object from the RCU root and then follows pointers stored in that object. The physical lease established for the root, however, does not transitively protect allocations reached through subsequent atomic links. Each such link may also be concurrently updated, and under relaxed memory a load may observe a stale pointer value from the link’s history. Therefore, before dereferencing a newly observed target, the proof must establish that the observation still refers to a protectable allocation and obtain a separate physical lease for that allocation.

We use the linked-list development to validate this additional proof obligation. Rather than verifying a complete linked-list implementation, the current mechanization isolates a single traversal step: starting from an already protected source allocation, it performs an acquire load of a weak atomic link, resolves the observed pointer to its allocation identity, excludes stale or already-removed link observations, extends the RCU guard’s protection state to the target allocation, and obtains the physical lease required to dereference it. Repeating such a step is the mechanism by which RCU protection can in principle be extended along a mutable pointer structure.

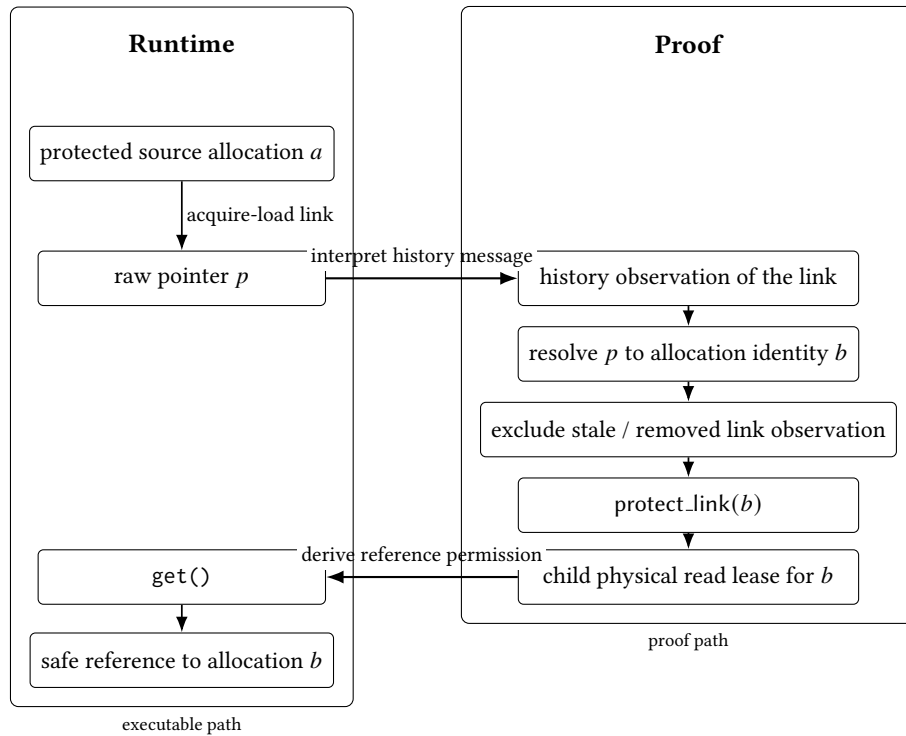


Figure 3: **Single-step extension of RCU protection across a weak atomic link.** Starting from a protected source allocation a , the reader performs an acquire load and obtains a raw pointer p . The proof resolves the corresponding history observation to allocation identity b , rules out stale or removed link observations, and extends the reader protection to b . Only after a physical read lease for b is obtained can `get()` derive the permission required to construct a safe Rust reference.

3.4 Safety property

Theorem 1. *Let $a \in \text{AId}$ be an allocation retired from a verified RCU root. If a verified reclamation callback recovers the exclusive ownership of a , then no active RCU reader holds a valid physical read lease for a . Moreover, exclusive ownership of a is recovered at most once.*

Proof. A callback can recover exclusive ownership only after receiving a matching $\text{ReclaimPermit}(a)$. By the grace-period rule, such a permit is produced only after every reader generation capable of producing a has been closed. Readers that started before retirement must therefore have released their leases, whereas readers starting after the relevant removal observation cannot newly protect a . Hence

$$\text{ReadLease}(a) \star \text{ReclaimPermit}(a) \vdash \perp.$$

The callback may therefore recover the physical ownership of a . Linearity of the reclaim permit and physical ownership prevents the ownership from being recovered twice. $\square$

4 Mechanization in Verus

We now refine the abstract protocol of Section 3 into the proof state and executable mechanisms used by Asterinas. The mechanization follows the same decomposition as the logical presentation. At the weak-memory layer, Verus’ native IRC11 resources represent atomic histories and execution-participant views. The RCU layer associates historical pointer values with stable allocation identities and derives physical read leases for allocations that a reader is permitted to access. For pointer structures, this protection is extended explicitly across weak atomic links rather than propagated transitively from the root. Finally, retirement certificates, per-CPU grace-period state, and proof-carrying callbacks connect the disappearance of all reader leases to recovery of exclusive physical ownership.

A central refinement boundary is the distinction between objective shared state and subjective execution state introduced in Section 3. Atomic histories, allocation metadata, and ownership information reside in shared invariants. In contrast, the weak-memory view of a running task must follow that task through atomic accesses and scheduler transitions. We therefore distribute proof state across the atomic objects, running-task context, RCU guards, CPU-local participants, and RCU monitor.

4.1 Native IRC11 Atomics and Execution Views

At the lowest layer, we reuse Verus’s native IRC11 implementation rather than introducing a second weak-memory model. For each atomic location ℓ , an $\text{AtomicPointsTo}\langle T \rangle$ resource owns a finite logical history

$$\text{Hist}_\ell : \mathbb{N} \rightarrow (T \times \text{ThreadView}),$$

whose entries correspond to the history messages described in Section 3. A ViewSeen resource represents a lower bound on the observations of the current execution participant. Thus, the histories and subjective views used by the RCU proof are the native IRC11 resources themselves than a separate RCU-specific encoding.

The current Verus IRC11 interface does not provide a weak-memory AtomicPtr abstraction. We therefore implement $\text{PAtomicWeakPtr}\langle T \rangle$ as a thin executable adapter around Rust’s $\text{AtomicPtr}\langle T \rangle$, with specifications stated directly in terms of the native IRC11 load, store, and update relations. The adapter supports the operations needed by the RCU implementation, including acquire and relaxed loads, release and relaxed stores, compare-and-exchange, and release swap. The executable atomic access and its corresponding logical history transition are treated as one atomic Verus operation and hence share a linearization point.

Execution views require a different treatment as they are subjective state belonging to an execution participant rather than an atomic location. In particular, the view used by one weak atomic operation cannot be reconstructed independently for the next operation; observations imported by an acquire must remain available to subsequent accesses.

We therefore make the task's IRC11 view part of the scheduler-controlled execution state. A scheduler-owned `ThreadViewToken` wraps the native `ViewSeen`. When a task is scheduled in, its task view is checked out and joined with the persistent view of the CPU on which it runs. While the task executes, this checked-out view is retained by its `RunningTaskContext`; during a preemption-disabled interval, `PreemptThreadViewSession` provides mutable access to the same underlying `ViewSeen`. Weak atomic operations borrow this resource rather than creating a fresh view for each access.

On schedule-out, the resulting observations are joined back into the scheduler-maintained task and CPU state. Consequently, an observation imported by an acquire load remains part of the execution participant's view across later RCU operations and scheduler transitions. This is the concrete realization of the subjective `Seen(V)` state introduced in Section 3.

4.2 RCU Root Protection and Physical Read Leases

The IRC11 layer establishes which historical pointer value a reader observes, but observing a raw pointer is not sufficient to dereference it. The RCU refinement must additionally identify the allocation represented by that history message, establish that the allocation may still be protected by the current reader, and derive a physical permission for accessing its contents. We perform these steps while refining the acquire load of the RCU root.

Each RCU root is associated with an atomic invariant whose state combines three components: the native IRC11 `AtomicPointsTo` resource, the RCU publication metadata, and the physical-permission state of registered allocations. Concretely, the invariant state has the form

$$\text{RootState} = \text{AtomicPointsTo} \times \text{RootGhost} \times \text{PermissionState}.$$

The publication metadata relates every non-null entry in the native atomic history to a stable allocation identity. In particular, a historical publication is represented by an `RcuPublishedObject` containing a domain-local allocation identifier a and its runtime address $\text{addr}(a)$. The association is indexed by the history timestamp rather than reconstructed from the raw address after the load. Consequently, two different allocations may reuse the same address without being confused by the proof.

The executable root-read path opens this invariant and performs the acquire. In addition to the raw pointer, the operation returns proof state describing the exact publication selected by the load. Schematically, its result⁴ is

$$\text{loadRoot}_{\text{acq}} \longrightarrow (p, t, \text{Published}(a, p), \text{BlockInfo}(a, p), \\ \text{CpuGuard}, \text{ReadLease}(a)).$$

For a non-null result, the selected IRC11 timestamp is therefore connected to both the raw pointer p and a particular allocation identity a . The returned `RcuBlockInfo` is persistent evidence that this allocation identity denotes the corresponding registered object; the `CpuRcuReadGuardToken` records the live RCU reader and its protection state.

The remaining obligation is to show that the selected allocation is protectable by the current reader. The reader guard records the set of allocations that had already expired at its starting view. Suppose that the allocation identity a returned by the root load were already in this expired set. The corresponding retirement fact would imply that the reader's starting view already observes the root-removal message for a .

⁴Through the `load_acquire_rcu_guarded_cpu` interface.

Let t_r be the first root-history timestamp after a was removed and let t be the timestamp selected by the acquire load. Since the reader already observes the removal, IRC11 coherence requires

$$t_r \leq t.$$

On the other hand, the root invariant records that a history message publishing allocation a must precede its removal:

$$\text{Hist}_\ell(t) = \langle \text{addr}(a), V_m \rangle \implies t < t_r.$$

The two inequalities are contradictory. Hence the allocation selected by a valid guarded root load is not expired at the reader's starting view and may be inserted into the guard's protection map.

Logical protection is then connected to physical memory ownership. The root invariant maintains an AId-indexed `RcuRootPermissionState`. For each registered allocation, this state contains a physical permission pool from which a live reader may obtain an `RcuRootReadLease`. The lease is indexed by the allocation identity rather than by its address and is additionally associated with the CPU reader generation, the reader's starting view, and the protected address. Thus, the lease records not only that some read permission exists, but also which live RCU reader justifies that permission.

The guarded root load splits such a lease for the selected allocation and stores it in the executable `RcuReadGuard`. Retaining the lease prevents the full physical permission from being recovered for reclamation while that reader remains active. When the guard is consumed, the lease is returned to the root permission state together with the corresponding CPU reader resource.

Finally, `RcuReadGuard::get()` turns the physical lease into the permission required by the Rust pointer abstraction. It does not construct a reference from the raw pointer alone. Instead, it borrows the physical permission stored in the lease,

$$\text{ReadLease}(a, P) \longrightarrow \text{RefPermission}(P),$$

through $P::\text{borrow_perm_as_ref_perm}$, and supplies the resulting permission to $P::\text{raw_as_ref}$. Hence the complete refinement path is

$$\begin{aligned} \text{IRC11 history message} &\longrightarrow \text{allocation identity} \\ &\longrightarrow \text{RCU protection} \\ &\longrightarrow \text{physical read lease} \\ &\longrightarrow \text{Rust reference.} \end{aligned}$$

The last step remains generic over the owning pointer type P . For example, an RCU cell containing a `Box<T>` yields a `BoxRef<T>`, while one containing an `Arc<T>` yields an `ArcRef<T>`; both may subsequently expose a shared reference to their target. In particular, the `Arc` case does not require incrementing the reference count merely to perform an RCU read.

4.3 Extending Protection Across Weak Atomic Links

The root-read refinement of the previous subsection establishes protection only for the allocation selected directly from the RCU root. This protection does not propagate transitively through pointers stored in that allocation. If a reader subsequently follows a concurrently updated atomic link, the value returned by the link load may come from an older entry in the link history. The mechanization must therefore establish, for each such step, that the observed pointer denotes the intended allocation and that the corresponding historical edge can still justify reader protection before the target is dereferenced.

We validate this obligation using a linked-list acceptance development. The current mechanization does not implement a complete executable linked list. Instead, the list topology is represented in ghost

state, including registered allocation identities, historical successor relations, and incoming edges, while an executable link is represented separately by a weak atomic pointer. This separation lets us isolate the proof obligations of one traversal step without yet committing to a concrete node layout or traversal loop.

At the proof level, a traversal reader carries more information than the root lease alone. The `RcuReadGuardToken` contains the base RCU guard together with a `SeenRemoved` snapshot recording the removal information visible to the reader. A protected source pointer is represented by `RcuProtectedPtr`, which records both its runtime address and stable allocation identity and relates them to the same reader state. Thus, the input to a traversal step is not merely a raw source pointer, but evidence that the source allocation has already been installed in the live guard’s protection map.

The concrete child-load path is implemented by `load_acquire_and_lease_cpu`. Starting from a `CpuRcuReadGuardToken` and a protected source, the operation performs an acquire load of the weak atomic link while opening the link invariant. In addition to the executable raw pointer, it retains the native IRC11 timestamp and a `LinkedListLinkObservation` describing the selected historical message. The proof relates that message to the allocation identity registered for the target and checks it against the authoritative traversal state. The underlying `protect_link` transition requires the observed edge to be compatible with the reader’s `SeenRemoved` state, preventing an edge that is already

Schematically, the successful non-null path is

$$\text{Protected}(a) \xrightarrow{\text{load}_{\text{acq}}(\ell)} (p, t, \text{LinkObs}) \xrightarrow{\text{protect_link}} \text{Protected}(b) * \text{ReadLease}(b),$$

where a is the already protected source allocation, p is the raw value read from the link, and b is the allocation identity associated with the selected history message. Importantly, b is obtained from the registered history metadata rather than reconstructed from the address p alone.

Logical protection of b is still not sufficient to dereference it. The same link-invariant opening therefore also interacts with the AID-indexed physical permission state and, for a live target, splits an `RcuRootReadLease` for the child allocation. The resulting lease is associated with the same RCU reader, CPU participation state, and starting view as the surrounding critical section. Consequently, child leases participate in the same reclamation-exclusion argument as leases obtained directly from an RCU root.

The child read guard uses this lease exactly as in the root case. `get()` borrows the physical permission contained in the lease, derives the pointer type’s `RefPermission`, and only then invokes the verified raw-pointer-to-reference interface. When the child guard is finished, the physical lease is returned to the permission pool and its saved CPU reader fraction is rejoined with the live RCU guard, while the updated guard, protected-child witness, and link observation remain available to the proof.

The resulting mechanization therefore separates the lifetime of the RCU critical section from the permissions needed for individual dereferences: one RCU guard may span multiple logical traversal steps, but each newly observed allocation requires its own history argument and physical read lease. The current linked-list development validates this single-link boundary, including publication, protection, unlink, retirement, and reclamation building blocks. It does not yet contain a concrete linked-list node representation or a verified traversal loop; the current child-load interface is also acceptance-specific and uses stronger preconditions than a general data-structure client.

4.4 Global Completion and Callback-Specific Reclamation Authority

A `GracePeriod` collects one such report from each online CPU. Its executable state contains a CPU mask, while the proof state maintains a matching set of reports and closed-generation resources. The tracked shadow of the CPU mask makes the executable completion test correspond to the logical condition

$$\text{dom}(\text{reports}) = \text{online_cpus}().^5$$

⁵No CPU hotplugs.

When the final CPU reports, the monitor converts the current grace period into a `CompletedGracePeriod`. This proof token records the completed epoch, the callback summaries in that batch, the set of reporting CPUs, the corresponding `RcuCpuQuiescentReports`, and one matching `CpuRcuClosedGeneration` for every reported CPU. For every callback C in the completed batch and every reported CPU c , completion establishes $C.V_r \preceq V_c \wedge C.\text{retired_record} \in K_c$. Hence a completed grace period is a batch-level fact: the same collection of CPU reports may justify all callbacks assigned to that epoch.

4.5 Retirement and Grace-Period Refinement

The reader-side refinement described above establishes that every dereference of an RCU-protected allocation is justified by a live CPU-local reader resource and a corresponding physical read lease. Reclamation requires the converse argument: before recovering exclusive ownership of a retired allocation, the monitor must establish that no reader capable of protecting that allocation can remain active. In Asterinas, this argument is implemented through CPU-local reader generations, scheduler quiescent points, and a global grace-period monitor.

A useful distinction is between two forms of generation used by the mechanization. The RCU monitor maintains a global grace-period epoch that identifies a batch of callbacks waiting for the same round of CPU quiescence reports.

Retirement and Grace-Period Assignment. When an allocation is removed from an RCU-protected structure, the retirement path first produces an `RcuCallbackSafety` certificate. The certificate identifies the RCU domain, the retired allocation, the removal observation associated with that retirement, and the observation registry. When the corresponding executable callback is admitted to the monitor, these facts are combined with the scheduler identity, the current weak-memory view, and a monitor epoch to form an `RcuCallbackSummary` that looks like

$$C = \langle S, D, a, Q, R, e, V_r \rangle,$$

where a is the stable allocation identity for the given atomic location, Q is the root- or link-removal observation, e is the grace-period epoch to which the callback is assigned, and V_r is the retirement view. Construction requires that Q is already observed by V_r . The summary therefore gives the monitor a stable logical identity for the callback of the type-erased executable context.

The monitor maintains a current grace period and a queue of callbacks for the next one. If the monitor is idle, the first callback starts a fresh grace period. If a grace period with epoch e is already in progress, newly admitted callbacks are placed in the next batch and assigned epoch $e + 1$. When the current grace period completes, that queued batch becomes the next current grace period. Thus, callbacks in one completed batch share a monitor epoch, while callbacks arriving during that batch are not justified by the reports already collected for it.

The monitor also maintains a monotonic set of persistent retirement facts called `RcuRetiredFacts`. This state records retirements that have already become visible through the monitor synchronization path. It is lazily copied into CPU-local evidence. Retaining this information is necessary because grace-period completion must establish not only that a CPU passed a quiescent point, but also that the corresponding weak-memory view had advanced far enough to observe the retirements covered by the callback batch.

CPU-Local Reader Generation. Each online CPU owns a persistent `CpuRcuParticipant`. This is a resource algebra-based proof-only token [9]. Starting an RCU read-side critical section splits a fractional `CpuRcuReaderFragment` from this participant. The fragment is retained by the executable RCU guard, and physical read leases derived by that guard retain corresponding CPU-generation witnesses. Consequently, the CPU cannot report the end of the current reader generation while any reader fragment

from that generation remains outstanding: producing a quiescent report requires recovering the complete participant fraction. At a scheduler quiescent point, the running-task context closes the current CPU-local generation. Abstractly, the proof transition has the form

$$\text{Participant}(c, g, V, K) \Rightarrow \text{Participant}(c, g + 1, V', K') \star \text{Closed}(c, g, V, K),$$

where c is the CPU identifier, g is the generation being closed, V is the weak-memory view at the quiescent boundary, and K is the set of retirement records known at that boundary. The concrete closed resource is `CpuRcuClosedGeneration`. The running-task context simultaneously advances its CPU RCU generation by one.

The monitor wraps this transition in an `RcuQuiescentContext`, which records the CPU, scheduler-owned participant identity, closed generation, and weak-memory view. It then records a persistent `RcuCpuQuiescentReport` together with the matching `CpuRcuClosedGeneration`. A report is therefore stronger than the statement that a CPU happened to execute a scheduler transition. For the callbacks currently covered by the grace period, it additionally establishes

$$V_r \preceq V_c \wedge \text{RetiredRecord}(C) \in K_c,$$

where V_c and K_c are respectively the report view and the `known_retired` set carried by the closed-generation resource. In our code, the `record_quiescent_state` requires both properties for every callback in the current batch before the CPU can be recorded as having reported.

The view-coverage obligation is connected to executable synchronization through the monitor lock. The lock-protected state maintains a proof-only `ThreadViewToken`. A lock holder joins the previously published lock view into the current `ViewSeen`, and before releasing the lock publishes its resulting observations back into the token. The retirement facts retained by the monitor are therefore observed by a task before that task constructs the quiescent context used for the report. In this way, the existing monitor lock provides the release/acquire synchronization needed to connect callback retirement views to later CPU reports without introducing additional executable synchronization solely for the proof.

The central property of `CpuRcuClosedGeneration` is that it classifies every reader that can coexist with the report. A reader from the generation just closed cannot coexist with the closed-generation resource, because the report could not have been created while its reader fraction remained live. Any reader that does coexist with the report must therefore belong to a strictly later generation; resource validity additionally gives that $g_c < g_{\text{reader}}$ as well as $V_c \preceq V_{\text{start}}$. This is the mechanism that connects CPU-local quiescence to the weak-memory argument required for RCU reclamation.

This yields two distinct cases for a reader of a retired allocation. A pre-existing reader is excluded because its generation has been closed and its reader resource cannot coexist with the closed-generation witness. A later reader may coexist with the witness, but its start view covers the report view. Since the report view covers the callback's removal observation, the later reader also observes that removal. The retirement record is consequently included in its `known-retired` state, and the allocation is classified as expired for that reader. Such a reader cannot newly establish protection for the same allocation. The mechanized `tracked_later_guard` lemma implements precisely this second case.

4.6 Proof-Carrying Callbacks

Delayed reclamation introduces a temporal verification boundary. When an object is retired, the executable callback already determines which object will eventually be destroyed. At that point, however, reclamation is not yet safe: readers that observed the object before its removal may still hold valid physical read leases. The authority to reclaim the object, represented by an `RcuReclaimPermit`, is produced only after the monitor has completed the corresponding grace period. Thus, delaying the callback is not by itself a sufficient proof of safety. The verification must preserve the relationship between the delayed computation, the retired allocation, and the future proof resource that will authorize reclamation.

We address this mismatch using a proof-carrying callback abstraction. Conceptually, a callback consists of a typed executable context and a logical invocation contract. The executable context captures the runtime state required by the delayed operation, while the contract specifies which proof resource must be supplied before the operation may run. In the RCU instance, this resource is an object-specific `RcuReclaimPermit`.

Directly representing this dependency using Asterinas’s conventional `Box<dyn FnOnce() + Send + 'static>` callback interface is not practical in the current Verus setup. In particular, Verus has limited support for reasoning about trait objects, while verified reclamation needs to transfer a proof-only argument to the callback at invocation time. We therefore construct callbacks from sized context types implementing the following verification-specific interface:

```

1 trait RawCallbackContextWithProof<S>: Send + 'static {
2     spec fn call_requires(&self, proof: S) -> bool;
3
4     fn run(self, proof: Tracked<S>)
5         requires self.call_requires(proof@);
6 }

```

While the concrete context type is known, Verus can establish a `call_requires` predicate relating the captured context to the proof required for invocation. The context is then erased into a special trait object `RawCallbackWithProof<S>`. Its executable representation contains a thin pointer to the heap-allocated context and the address of a monomorphized runner; the parameter `S` has no runtime proof payload. Crucially, construction establishes that the erased callback’s `call_requires` predicate is exactly the invocation contract of the original typed context. Type erasure therefore removes the Rust context type without erasing the logical precondition under which the context may be invoked.

For RCU, we add a second wrapper around this generic mechanism. A queued callback has the form

```

1 struct RcuCallback {
2     raw: RawCallbackWithProof<RcuReclaimPermit>,
3     summary: Ghost<RcuCallbackSummary>,
4     safety: Tracked<RcuCallbackSafety>,
5 }

```

The `RcuCallbackSafety` certificate can only be obtained from the retirement path. When the callback is inserted into the monitor, `RcuCallback::from_raw` combines this certificate with the retirement epoch and retirement view to construct an `RcuCallbackSummary`. The summary records the RCU domain, allocation identity, removal observation, retirement-observation registry, scheduler, retirement epoch, and retirement view. It therefore provides a stable logical identity for the reclamation operation even though the concrete callback context has been type-erased.

The reclaim permit is deliberately not stored in the queued callback. It does not yet exist when the callback is constructed. Instead, after the grace period completes, the monitor combines the callback’s retirement evidence with closed-generation evidence from every relevant online CPU to produce an object-specific `RcuReclaimPermit`. A permit authorizes a callback only when its summary matches the callback’s recorded identity and the CPU reports establish the required weak-memory view coverage. The verified invocation boundary therefore has the form

$$\text{RcuCallback}(C) \star \text{RcuReclaimPermit}(C) \vdash \text{run}(C).$$

At `call_once`, the monitor first proves that the permit authorizes the callback summary, which in turn establishes the `call_requires` condition of the erased `RawCallbackWithProof`. The permit is

then passed across the type-erasure boundary to the original typed callback context. Thus, the proof argument does not become part of the ordinary callback ABI: the Rust context type is erased when the callback is queued, while the logical identity and invocation contract remain available until grace-period completion.

For an RCU destruction callback, invocation is still not equivalent to unconditionally freeing the raw pointer. The typed callback combines the matching reclaim permit with its unique reclamation claim and opens the RCU permission invariant. This recovers the complete physical $P::\text{Permission}$ for the retired allocation. Only then can the callback reconstruct the owning pointer using $P::\text{from_raw}$ and allow it to be destroyed. Hence the complete reclamation path is

$$\begin{aligned} \text{retirement} &\longrightarrow \text{RcuCallbackSafety} \longrightarrow \text{type-erased queued callback} \\ &\longrightarrow \text{grace-period completion} \longrightarrow \text{RcuReclaimPermit} \\ &\longrightarrow \text{physical ownership recovery} \longrightarrow \text{owner reconstruction and destruction.} \end{aligned}$$

The raw data-pointer and runner erasure is implemented behind trusted `external_body` functions, and the one-shot invocation discipline remains an explicit language-integration boundary discussed in

5 Lessons Learned

The mechanization exposed several design constraints that are less visible in an abstract presentation of RCU, but become unavoidable once the proof is connected to executable kernel code.

Weak-memory views are execution state. A weak-memory view cannot be reconstructed independently for each atomic operation. An acquire load changes what the executing participant has observed, and those observations may later be needed to justify protection or reclamation. Consequently, the subjective IRC11 view must follow the execution participant across successive atomic accesses and scheduling boundaries. In our refinement, this required integrating the native `ViewSeen` with scheduler-controlled task and CPU state rather than treating it as local ghost state of the RCU object. This experience suggests that verification of relaxed-memory kernel code may require reasoning about the scheduler even when scheduling is not part of the abstract concurrent algorithm.

Allocation identity must be independent of pointer values. Relaxed memory makes historical pointer values part of normal execution: a load may legitimately observe an older message from an atomic history. Such a message can remain logically relevant even after the corresponding allocation has been retired, and a physical address may later be reused by a different allocation. We therefore found it necessary to distinguish stable allocation identities from both raw addresses and atomic-history timestamps. The atomic history records which allocation identity a pointer message denotes, while protection, retirement, and reclamation are indexed by that identity. This separation makes address reuse an explicit part of the proof rather than an implicit assumption about pointer values.

RCU protection and memory permission are different resources. A logical RCU guard can establish that an allocation is protected from reclamation, but this fact alone is not sufficient to construct a Rust reference. The executable refinement must additionally provide physical permission to access the allocation. We therefore introduced explicit read leases that connect RCU protection to the pointer-specific permission needed by Rust reference construction. This distinction also explains why protection does not propagate automatically through a pointer structure: one RCU critical section may span several traversal steps, but each newly observed allocation requires its own history argument and physical read lease before dereference.

Quiescence alone is not enough under relaxed memory. The grace-period proof revealed two logically separate requirements. First, a CPU must close the reader generation that could contain readers predating retirement. Second, the weak-memory view associated with that quiescent report must cover the retirement and removal observations needed by the callback. The first condition rules out surviving old readers, while the second constrains readers that start afterwards. Our implementation therefore tracks both per-CPU reader generations and their associated IRC11 views. This distinction is important: a lifetime argument showing that old readers have finished does not, by itself, establish the visibility facts required for safe reclamation under relaxed memory.

Delayed execution may require delayed proof transfer. Reclamation callbacks expose a temporal mismatch between executable state and proof authority. The callback is created when an object is retired, but the authority to reclaim that object does not exist until the grace period has completed. Hence the callback cannot simply capture all of the resources that will eventually be needed to run it. Our proof-carrying callback interface instead preserves an invocation contract across type erasure and supplies the `RcuReclaimPermit` only at the eventual invocation boundary. This pattern applies more generally to deferred kernel operations whose executable work is queued before the proof authority required to perform that work becomes available.

Taken together, these lessons suggest that the main difficulty in refining RCU to a verified kernel implementation does not lie in the abstract read-side protocol alone. The proof must also connect weak-memory observations to scheduler state, historical pointer values to stable allocation identities, logical protection to physical memory ownership, and grace-period completion to delayed executable work. Making these interfaces explicit helped localize the remaining trusted boundaries and also made the root-level reclamation argument reusable when reasoning about internal atomic links and more complex RCU-protected data structures.

6 Conclusion

In this report, we presented a verification of RCU-based memory reclamation in the Asterinas kernel under relaxed memory. Our development combines an Iris-style concurrent separation-logic interface with Verus’s native IRC11 weak-memory model, connecting abstract RCU reasoning to executable kernel mechanisms. In particular, we showed how a historical atomic-pointer observation is associated with a stable allocation identity, extended into logical RCU protection, and refined into a physical read lease that justifies the construction of a Rust reference. On the writer side, retirement, per-CPU quiescent-state tracking, weak-memory view propagation, and proof-carrying callbacks are connected to produce an object-specific reclamation permit and eventually recover exclusive physical ownership.

The resulting mechanization verifies an end-to-end reclamation path for RCU root pointers, including weak atomic publication, reader protection, physical leases, grace-period completion, and delayed destruction. We additionally developed a linked-list acceptance model that validates the weak-memory proof obligations required to extend protection across one mutable atomic link, including stale-link exclusion and per-target lease acquisition. This development establishes the building blocks needed for verified traversal, but does not yet constitute an end-to-end verified linked-list implementation.

More broadly, the development shows that verifying RCU under relaxed memory requires reasoning simultaneously about logical reader protection, physical memory ownership, allocation identity, execution-participant views, and asynchronous reclamation authority. Making these resources explicit exposes the assumptions hidden by a purely operational description of RCU and provides a path toward integrating verified reclamation with more complex RCU-protected data structures in Asterinas.

References

- [1] Stephen Brookes and Peter W O’Hearn. Concurrent separation logic. *ACM SIGLOG News*, 3(3):47–65, 2016.
- [2] Jaehwang Jung, Sunho Park, Janggun Lee, Jeho Yeon, and Jeehoon Kang. Verifying general-purpose rcu for reclamation in relaxed memory separation logic. *Proceedings of the ACM on Programming Languages*, 9(PLDI):1–25, 2025.
- [3] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of functional programming*, 28:e20, 2018.
- [4] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. *ACM SIGPLAN Notices*, 50(1):637–650, 2015.
- [5] Jan-Oliver Kaiser, Hoang-Hai Dang, Derek Dreyer, Ori Lahav, and Viktor Vafeiadis. Strong logic for weak memory: Reasoning about release-acquire consistency in iris. In *31st European Conference on Object-Oriented Programming (ECOOP 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017.
- [6] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):286–315, 2023.
- [7] Paul E McKenney and John D Slingwine. Read-copy update: Using execution history to solve concurrency problems. In *Parallel and Distributed Computing and Systems*, volume 509518, pages 509–518, 1998.
- [8] Glen Mével and Jacques-Henri Jourdan. Formal verification of a concurrent bounded queue in a weak memory model. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–29, 2021.
- [9] Luc Moreau and Christian Queinnec. Resource aware programming. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 27(3):441–476, 2005.
- [10] Yuke Peng, Hongliang Tian, Junyang Zhang, Ruihan Li, Chengjun Chen, Jianfeng Jiang, Jinyi Xian, Xiaolin Wang, Chenren Xu, Diyu Zhou, et al. {ASTERINAS}: A linux {ABI-Compatible},{Rust-Based} framekernel {OS} with a small and sound {TCB}. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 307–323, 2025.
- [11] John C Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings 17th annual IEEE symposium on logic in computer science*, pages 55–74. IEEE, 2002.
- [12] Abhayendra Singh, Satish Narayanasamy, Daniel Marino, Todd Millstein, and Madanlal Musuvathi. End-to-end sequential consistency. *ACM SIGARCH Computer Architecture News*, 40(3):524–535, 2012.
- [13] Joseph Tassarotti, Derek Dreyer, and Viktor Vafeiadis. Verifying read-copy-update in a logic for weak memory. *ACM SIGPLAN Notices*, 50(6):110–120, 2015.